# INJAAZ

Full Technical Documentation

Methods · Functions · Flows · Code References

Generated: 09 April 2026

Version 1.0 — Internal Reference

# Table of Contents

1

# Application Overview & Architecture

Injaaz is a **modular monolith** built with **Python / Flask**. One deployable application exposes many Flask blueprints — each covering a business domain — that all share a single SQLAlchemy database, a common authentication layer, and shared utility libraries under **common/**.

## Modules and URL prefixes

| Module / Blueprint | URL Prefix | File |
|---|---|---|
| Authentication | /api/auth | app/auth/routes.py |
| Admin | /api/admin | app/admin/routes.py |
| Workflow | /api/workflow | app/workflow/routes.py |
| Reports API | /api/reports | app/reports_api.py |
| HVAC & MEP | /hvac-mep | module_hvac_mep/routes.py |
| Civil Works | /civil | module_civil/routes.py |
| Cleaning | /cleaning | module_cleaning/routes.py |
| Inspection | /inspection | module_inspection/routes.py |
| HR | /hr | module_hr/routes.py |
| Procurement | /procurement | module_procurement/routes.py |
| MMR / Reporting | /admin/mmr | module_mmr/routes.py |
| Business Dev | /bd | app/bd/routes.py |
| DocHub | /api/docs | app/docs/routes.py |

## Blueprint registration pattern

Each blueprint is imported in a guarded **try/except** block at the top of **Injaaz.py**. If an import fails (e.g. a missing dependency), the app still starts and all other modules remain available. The failed module is silently absent from the URL space until fixed.

```
# Safe import pattern used for every blueprint
hr_bp = None
try:
from module_hr.routes import hr_bp
except Exception as e:
logger.exception('Could not import hr_bp: %s', e)
# Registration inside create_app()
if hr_bp:
app.register_blueprint(hr_bp, url_prefix='/hr')
```

2

# Application Factory — Injaaz.py

**create_app()** is the Flask application factory. It is the single entry point that wires together every component. Nothing is initialised at module level — all setup happens inside this function, which means the app can be instantiated multiple times with different config (e.g. test suite vs production).

## What create_app() does — in order

| Step | What happens |
|---|---|
| Load config | Reads every uppercase variable from config.py and sets it on app.config. |
| Validate config | Calls common.config_validator.validate_config(app) — raises RuntimeError on critical failures so a misconf |
| Init DB + Bcrypt | db.init_app(app) and bcrypt.init_app(app) — SQLAlchemy and password hashing. |
| Init Flask-Migrate | Migrate(app, db) — registers 'flask db upgrade/migrate' CLI commands for schema versioning. |
| Init JWTManager | Attaches JWT error handlers: missing token → redirect or 401; expired → redirect or 401; invalid → same; t |
| JWT blocklist check | token_in_blocklist_loader — every request queries the Session table by JTI. If the session row is missing or |
| DB startup migration | Runs db.create_all() then inspects existing columns; adds any missing columns with ALTER TABLE for bac |
| Default admin seed | If no admin user exists, creates one with a generated or configured password and logs it prominently so ope |
| Register blueprints | Registers all successfully imported blueprints with their URL prefixes. |
| Register page routes | Attaches non-blueprint routes: /, /login, /dashboard, /about, /admin/*, etc. as plain view functions. |
| PWA + static routes | Serves manifest.json, service-worker.js, offline.html from the static folder. |
| Error handlers | 404 and 500 handlers return JSON for /api/* paths and HTML pages otherwise. |
| Health endpoint | GET /health — no auth required; returns {status:ok, database:ok}. |

3

# Configuration — config.py

All runtime settings live in **config.py** as uppercase module-level variables, driven by **os.getenv()** calls. The app factory loads them automatically. Sensitive values must be set as environment variables — never hard-coded.

| Variable | Purpose | Default |
|---|---|---|
| SECRET_KEY | Flask session signing | change-me-in-production |
| JWT_SECRET_KEY | JWT signing secret | change-me-jwt-secret |
| DATABASE_URL | Full DB connection string | SQLite in dev, required in prod |
| REDIS_URL | Rate limiter / background queue | None (optional) |
| CLOUDINARY_* | Cloud file storage credentials | None (required in prod) |
| MAILJET_* / BREVO_* | Email API keys | None (optional) |
| JWT_ACCESS_TOKEN_EXPIRES | Access token lifetime | 1 hour |
| JWT_REFRESH_TOKEN_EXPIRES | Refresh token lifetime | 7 days |
| MAX_CONTENT_LENGTH | Max total upload size | 100 MB |
| GENERATED_DIR | Where uploads and jobs are stored | ./generated |
| FLASK_ENV | development / production | development |

4

# Database Models — app/models.py

All ORM models are defined in **app/models.py** using Flask-SQLAlchemy. The file also exports **db** (SQLAlchemy instance) and **bcrypt** (Bcrypt instance) used across the application.

### User

stores user account: username, email, bcrypt password hash, role, designation, is_active, per-module flags (access_hvac, access_civil, access_cleaning, access_hr, access_procurement_module), default_signature, default_comment, last_login.

Key methods:

- set_password(password) — hashes and stores password
- check_password(password) → bool — verifies against hash
- has_module_access(module) → bool — respects admin bypass
- to_dict() → dict — API serialisation

### Submission

central record for every form submission across all modules. Key fields: submission_id (unique slug), user_id, module_type, site_name, visit_date, status, workflow_status, form_data (JSON), created_at, updated_at. Holds FK columns for every approval participant (supervisor_id, operations_manager_id, general_manager_id …).

Key methods:

- to_dict(include_form_data, include_latest_job) → dict

### Job

tracks a background report-generation task. Fields: job_id, submission_id, status (pending/processing/completed/failed), progress (0–100), result_data (JSON with file URLs), error_message, started_at, completed_at.

Key methods:

- to_dict() → dict

### File

metadata for an uploaded file: submission_id, file_type, cloud_url, filename, file_size.

Key methods:

- to_dict() → dict

### AuditLog

immutable event record: user_id, action, resource_type, resource_id, ip_address, user_agent, details (JSON).

Key methods:

- to_dict() → dict

### Session

JWT session row: user_id, token_jti (unique), expires_at, is_revoked.

Key methods:

- to_dict() → dict

### Notification

in-app notification: user_id, title, message, notification_type, submission_id, is_read.

Key methods:

- to_dict() → dict

### Device

tracked device record used by admin module.

Key methods:

- to_dict() — includes human-readable 'last_active' string

### BDProject / BDFollowUp / BDContact / BDActivity

Business Development entities with relational links.

Key methods:

- to_dict() for each — BD card-style payload

### AdminPersonalProject / AdminPersonalProgressStep

Personal project tracking with checklist steps and progress %.

Key methods:

- to_dict() — computes progress percentage from steps

### DocHubDocument / DocHubAccess

Document hub storage and per-user access flags.

Key methods:

- to_dict() — includes content/refs for content-type docs

### MmrChargeableConfig

Single-row JSON config for MMR chargeable rules.

5

# Common Utilities — common/

The **common/** package is imported by every module. It prevents code duplication and ensures consistent behaviour across the application.

## config_validator.py

| Function / Class | Description |
|---|---|
| validate_config(app) → (bool, list) | Checks SECRET_KEY, JWT_SECRET_KEY and DATABASE_URL for unsafe values in produ |

## jwt_session.py

| Function / Class | Description |
|---|---|
| sync_access_session_row(jti, jwt_payload) | Ensures a UserSession row exists for the given JTI. Used by the JWT blocklist loader when a token |

## error_responses.py

| Function / Class | Description |
|---|---|
| error_response(message, status_code, error_code, details) | Returns JSON {success:false, error:…, error_code:…, details:…}. Used by every API route for |
| success_response(data, message, status_code) | Returns JSON {success:true, data:…, message:…}. |
| handle_exceptions(f) | Decorator that wraps a route in try/except and returns error_response(500) on unhandled exce |

## utils.py

| Function / Class | Description |
|---|---|
| random_id(prefix='') → str | UUID4 hex with optional prefix — used to generate submission_id and job_id. |
| ensure_dir(path) | os.makedirs with exist_ok=True. |
| write_job_state / read_job_state | Read/write a JSON job-state file in JOBS_DIR (file-based fallback when DB unavailable). |
| mark_job_started / update_job_progress / update_job_done | Update job row in DB and write state file. |
| upload_file_to_cloud(file_path, public_id) → url | Upload a local file to Cloudinary and return the secure URL. |
| upload_base64_to_cloud(data_url, public_id) → url | Decode a base64 data URL and upload to Cloudinary. |
| get_image_for_pdf(url_or_b64) → BytesIO \| None | Download image from URL (with retry) or decode base64, returning bytes for ReportLab. |
| is_path_safe_for_directory(base, path) → bool | Prevent path traversal by confirming resolved path is inside base directory. |

## db_utils.py

| Function / Class | Description |
|---|---|
| create_submission_db(…) → Submission | Insert a new Submission row; returns the committed object. |
| create_job_db(submission_id) → Job | Insert a pending Job row for a submission. |
| update_job_progress_db(job_id, progress, message) | Set Job.progress and update timestamp. |
| complete_job_db(job_id, result_data) | Set Job.status='completed', store result_data JSON. |
| fail_job_db(job_id, error_message) | Set Job.status='failed', store error string. |
| get_job_status_db(job_id) → dict | Return serialised Job or {status:'not_found'}. |

## module_base.py

| Function / Class | Description |
|---|---|
| process_report_job(sub_id, job_id, app, module_name, create_pdf_report, create_excel_report, label) | Shared background job runner for HVAC, Civil, and Cleaning. Steps: mark started – |

## security.py

| Function / Class | Description |
|---|---|
| sanitize_filename(name) → str | Remove non-alphanumeric chars except dots and dashes. |
| safe_path_join(base, *parts) → str \| None | Return joined path only if it stays inside base; returns None on traversal attempt. |
| validate_json_request(required_fields) → dict or (…, 400) | Raise 400 if required JSON fields are missing. |
| validate_file_upload(file, allowed_exts, max_mb) → (bool, msg) | Check extension and size. |
| log_security_event(event, user_id, details) | Write a WARNING-level security audit entry. |
| check_cloudinary_configured() → bool | Return True if all three Cloudinary env vars are set. |

## email_service.py

| Function / Class | Description |
|---|---|
| send_email(to, subject, body_html, body_text, attachments) → (bool, msg) | Try all email providers in order. Tries Brevo HTTP API → Mailjet HTTP API → SMTP in order, |
| send_password_reset_email(user, temp_password) | Composes and sends a password reset email using send_email. |
| is_email_configured() → bool | Returns True if at least one email provider is configured. |

## cache.py

| Function / Class | Description |
|---|---|
| get_redis_connection() → Redis \| None | Return Redis client from REDIS_URL, or None if unavailable. |

| Function / Class | Description |
|---|---|
| cached(key, ttl=3600) → decorator | Cache the return value of the decorated function in Redis for ttl seconds. |
| invalidate_cache(key) | Delete a key from Redis. |

## retry_utils.py

| Function / Class | Description |
|---|---|
| upload_to_cloudinary_with_retry(data, public_id, retries=3) → dict | Cloudinary upload with exponential backoff on transient failures. |
| fetch_url_with_retry(url, retries=3) → bytes | HTTP GET with retry. |

## validation.py

| Function / Class | Description |
|---|---|
| HVACItemSchema, CivilWorkItemSchema, ElectricalItemSchema, ... | Field-level schemas for each work item type (Marshmallow). |
| validate_request(schema_class, data) → (errors_dict \| None, validated_data \| None) | Run schema validation; return validation errors or the clean object. |

## workflow_notifications.py

| Function / Class | Description |
|---|---|
| send_team_notification(submission, action, user, app) | Send in-app (and optionally email) notification to the relevant team members when a workfl |

6

# Authentication — app/auth/routes.py

Blueprint prefix: **/api/auth**. Handles registration, login, token refresh, logout, profile, default signature, and password change.

| Function | Method | Path | Auth | Description |
|---|---|---|---|---|
| validate_email(email) | — | — | — | Regex email check used before DB query. |
| validate_password(pw) | — | — | — | Min 8 chars, upper, lower, digit. Returns (ok, msg). |
| log_audit(…) | — | — | — | Writes AuditLog row with IP and user-agent. |
| register | POST | /api/auth/register | None | Create user; validates uniqueness, hashes pw. |
| login | POST | /api/auth/login | None (rate-limited) | Verify creds → issue access+refresh JWT → Session |
| refresh | POST | /api/auth/refresh | Refresh JWT | Mint new access token from valid refresh token. |
| logout | POST | /api/auth/logout | JWT | Revoke Session row, clear cookies. |
| get_current_user | GET | /api/auth/me | JWT | Return caller's User.to_dict(). |
| update_signature_default | POST | /api/auth/signature-default | JWT | Store base64 signature + default comment on User. |
| change_password | POST | /api/auth/change-password | JWT | Verify old pw, update hash, revoke all sessions (force |

## Login flow — step by step

```
POST /api/auth/login { username, password }
1. Rate limit check (Flask-Limiter, 5/min)
2. User.query.filter( username OR email == input ).first()
3. user.check_password(password) → bcrypt.check_password_hash
4. user.is_active check
5. create_access_token(identity=str(user.id)) # JWT_ACCESS_TOKEN_EXPIRES
6. create_refresh_token(identity=str(user.id)) # JWT_REFRESH_TOKEN_EXPIRES
7. Session(token_jti=jti, expires_at=exp) → db.session.commit()
8. set_access_cookies() + set_refresh_cookies() # HTTP-only
9. Return { access_token, refresh_token, user:{…}, requires_password_change }
```

7

# Admin Module — app/admin/routes.py

Blueprint prefix: **/api/admin**. All routes require **JWT + role=admin**. Covers user management, submission editing, MMR config, BD projects, devices, DocHub access control, and a personal progress tracker.

## User management functions

| Function | Path | Description |
|---|---|---|
| list_users | /users | List all users; supports query filters. |
| get_user | /users/<id> | Single user detail + recent audit entries. |
| update_user | PUT /users/<id> | Change email, full_name, role, designation. |
| reset_user_password | /users/<id>/reset-password | Generate temp password, email it, mark password_changed=False. |
| toggle_user_active | /users/<id>/toggle-active | Flip is_active; revokes sessions if deactivating. |
| update_user_access | /users/<id>/update-access | Set any combination of module access flags. |
| set_user_designation | PUT /users/<id>/designation | Set GM/HR/supervisor/etc. designation. |
| delete_user | DELETE /users/<id> | Delete user and associated records. |
| get_user_activity | /users/<id>/activity | AuditLog entries for user. |
| get_users_by_designation | /users/by-designation/<d> | Filter users by designation. |

## Other admin functions

| Function | Description |
|---|---|
| dashboard_overview | Aggregate stats: user count, submissions by status, pending counts across all modules. |
| get_workflow_stats | Per-module submission counts, completion rates, recent activity. |
| update_submission | Admin-only direct edit of any submission's form_data or metadata. |
| close_submission | Admin force-close a submission at any workflow stage. |
| mmr_chargeable_config GET/PUT | Load or replace the MMR chargeable-classification JSON rules. |
| mmr_chargeable_preview | Test a set of rows against the chargeable rules and return classification preview. |
| bd_dashboard_data | BD projects, follow-ups, contacts, and statistics in one payload. |
| bd_create/update_project | CRUD for BD projects. |
| bd_import_projects_excel | Parse an Excel upload and bulk-insert BD projects. |
| list/create_device | Device register management. |
| import_devices_excel | Bulk device import from Excel. |

| Function | Description |
|---|---|
| list_dochub_access_users | Return per-user DocHub access flags. |
| set_dochub_user_access | Grant or revoke DocHub access for a specific user. |
| personal_progress_* | CRUD for admin's personal task/project checklist with step tracking. |

8

# Workflow Engine — app/workflow/routes.py

Blueprint prefix: **/api/workflow**. All routes use JWT. This module is the multi-stage approval backbone used by Civil, HVAC, Cleaning, and Inspection submissions (HR has its own separate workflow in module_hr).

## Workflow stages

| Stage (workflow_status) | Actor | Endpoint |
|---|---|---|
| submitted | Submitter (auto) | POST /submit |
| operations_manager_review | Operations Manager | POST /approve-ops-manager |
| bd_procurement_review | BD / Procurement | POST /approve-bd or /approve-procurement |
| general_manager_review | General Manager | POST /approve-gm |
| completed | System | After GM approval |
| rejected | Any approver | POST /reject |

## Key helper functions (non-route)

| Helper | Description |
|---|---|
| _dashboard_persona(user) | Returns 'admin'/'manager'/'user' to tailor dashboard data. |
| _hero_metrics_for_user(user) | Builds the four headline metric cards shown on the workflow dashboard. |
| _forms_needing_completion_count(user) | Count of submissions where the current user is a required next approver. |
| _filter_inspection(submissions, user) | Visibility filter for inspection submissions. |
| _filter_hr(submissions, user) | Visibility filter for HR submissions. |
| _completion_rate_pct(user) | Percentage of completed vs total relevant submissions. |
| _time_ago(dt) | Human-readable relative time string ('2 hours ago'). |
| _signature_url_from_field(field_value) | Extract a usable URL or base64 string from a stored signature field. |
| can_edit_submission(user, submission) | Returns True if the user is allowed to edit the given submission. |
| _admin_reviewers_for_history(submission) | Resolve the list of approvers who should appear in history view. |

## Approval route pattern (example)

```
POST /api/workflow/submissions/<submission_id>/approve-ops-manager
```

```
1. @jwt_required() — validate token
2. Verify user.designation == 'operations_manager' or role == 'admin'
3. submission = Submission.query.filter_by(submission_id=…).first()
4. Verify submission.workflow_status == 'submitted'
```

```
5. form_data = dict(submission.form_data or {}) # copy for ORM detection
6. form_data['operations_manager_signature'] = request.json.get('signature')
7. form_data['operations_manager_comments'] = request.json.get('comments')
8. submission.workflow_status = 'bd_procurement_review'
9. submission.operations_manager_id = user.id
10. db.session.commit()
11. send_team_notification(submission, user, 'approved')
12. Return {success:true, message:'Forwarded to BD/Procurement'}
```

9

# HR Module — module_hr/routes.py

Blueprint prefix: **/hr**. All routes require JWT. The HR module handles 11 form types through a two-stage approval chain: Employee → HR → General Manager.

## Helper functions

| Helper | Description |
|---|---|
| get_current_user() | Resolve User from JWT identity string. |
| _hr_form_context(user) | Returns {is_hr, is_gm} booleans — used to show/hide fields in templates. |
| create_notification(user_id, title, message, type, submission_id) | Insert a Notification row for one user. |
| get_form_type_display(module_type) | Maps 'hr_leave_application' → 'Leave Application'. |

## Page routes (HTML views)

| Handler | Path | Purpose |
|---|---|---|
| hr_dashboard | /hr/ | Main HR dashboard page. |
| my_requests | /hr/my-requests | Employee's own submission history. |
| leave_application_form | /hr/leave-application-form | Leave form HTML. |
| commencement_form | /hr/commencement-form | Commencement form HTML. |
| duty_resumption_form | /hr/duty-resumption-form | Duty resumption HTML. |
| contract_renewal_form | /hr/contract-renewal-form | Contract renewal HTML. |
| performance_evaluation_form | /hr/performance-evaluation-form | Evaluation HTML. |
| grievance_form | /hr/grievance-form | Grievance HTML. |
| interview_assessment_form | /hr/interview-assessment-form | Interview assessment HTML. |
| passport_release_form | /hr/passport-release-form | Passport release HTML. |
| staff_appraisal_form | /hr/staff-appraisal-form | Staff appraisal HTML. |
| station_clearance_form | /hr/station-clearance-form | Station clearance HTML. |
| visa_renewal_form | /hr/visa-renewal-form | Visa renewal HTML. |
| pending_review | /hr/pending-review | HR review queue. |
| gm_approval | /hr/gm-approval | GM approval queue. |
| approved_forms | /hr/approved-forms | Approved submissions archive. |
| hr_print | /hr/print/<id> | Print-ready HTML view of a submission. |

| Handler | Path | Purpose |
|---|---|---|
| hr_download_docx | /hr/download-docx/<id> | Stream DOCX file download. |
| hr_download_pdf | /hr/download-pdf/<id> | Stream PDF file download. |

## API routes (JSON)

| Handler | Method | Path | Who can call | Description |
|---|---|---|---|---|
| submit_hr_form | POST | /hr/api/submit | All authenticated users | Create Submission (workflow_status=hr_review |
| get_my_submissions | GET | /hr/api/my-submissions | All | Return caller's own HR submissions. |
| get_user_permissions | GET | /hr/api/user-permissions | All | Return {can_review_hr, can_approve_gm, is_a |
| get_pending_hr_review | GET | /hr/api/pending-hr-review | HR / Admin | Submissions with workflow_status=hr_review. |
| get_pending_gm_approval | GET | /hr/api/pending-gm-approval | GM / Admin | Submissions with workflow_status=gm_review. |
| get_approved_hr_submissions | GET | /hr/api/approved-hr-submissions | HR / GM / Admin | Submissions with workflow_status=approved. |
| hr_approve | POST | /hr/api/hr-approve/<id> | HR / Admin | Add hr_signature, hr_comments, optional form |
| hr_reject | POST | /hr/api/hr-reject/<id> | HR / Admin | Reject with reason; notify submitter. |
| gm_approve | POST | /hr/api/gm-approve/<id> | GM / Admin | Add gm_signature, gm_comments; mark appro |
| gm_reject | POST | /hr/api/gm-reject/<id> | GM / Admin | Reject with reason; notify submitter. |
| get_hr_submissions | GET | /hr/api/submissions | HR / GM / Admin | All HR submissions with filters. |
| get_notifications | GET | /hr/api/notifications | All | Notification list for current user. |
| get_unread_count | GET | /hr/api/notifications/unread-count | All | Integer count of unread notifications. |
| mark_notification_read | POST | /hr/api/notifications/<id>/read | All | Mark one notification as read. |
| mark_all_notifications_read | POST | /hr/api/notifications/mark-all-read | All | Mark all as read. |

## HR approval — code pattern

```
# hr_approve endpoint (key logic only)
form_data = dict(submission.form_data or {}) # copy — critical for ORM JSON detection
form_data['hr_reviewed_by_id'] = user.id
form_data['hr_reviewed_by_name'] = user.full_name
form_data['hr_reviewed_at'] = datetime.now().isoformat()
form_data['hr_comments'] = data.get('comments', '')
form_data['hr_signature'] = data.get('signature', '')
for k, v in (data.get('form_data_hr') or {}).items():
form_data[k] = v # e.g. hr_balance_cf, hr_checked
submission.form_data = form_data
submission.workflow_status = 'gm_review'
db.session.commit()
# then notify GM users via create_notification()
```

*Important: Always assign a new dict copy before mutating form_data. SQLAlchemy's JSON column type does not track in-place mutations; without a new dict assignment the changes are silently lost on commit.*

10

# HR PDF Builder — module_hr/hr_pdf_builder.py

Builds professional A4 PDFs using **ReportLab** directly — no intermediate format. Each form type has a dedicated _**build_*** function that constructs the exact table layout of the official form.

## Shared building blocks

| Function | Description |
|---|---|
| _fmt(v) | Format any value as string; returns em-dash for empty/None. |
| _fd(fd, key) | Safe dict get with _fmt fallback. |
| _sig_to_image(data_url, w_mm, h_mm) | Decode base64 data URL → ReportLab Image; optionally uses Pillow to knock out white background. |
| _get_styles() | Return dict of named ParagraphStyles (LABEL, VALUE, HEADER, SECTION, etc.). |
| _header_table(title, submission_id, doc_no) | Render the Injaaz logo, form title, and reference bar at the top of every PDF. |
| _section_bar(title) / _section_bar_colored(title, bg) | Coloured full-width section header row. |
| _data_table(rows, col_widths) | Generic label/value two-column table. |
| _form_fields(pairs) | Shortcut to build a _data_table from (label, value) pairs. |
| _long_field(label, value) | Full-width text area row. |
| _rating_table(rows) | 5-column rating table (criterion + score 1–5). |
| _checklist_table(items, checked_set) | Checkbox grid — ticked or empty per item. |
| _signature_block(signers) | Multi-column row with name label, signature image, and date. |
| _footer_block(doc_no, issue_date) | Document control footer. |
| _fstyles / _flv / _fsec / _fchk / _fsig / _ftable | Compact inline builders for the later-form template style. |
| _leave_tick(fd, key, label) | Returns '[✓] Label' or '[ ] Label' for leave type checkbox rows. |

## Form-specific build functions

| Function | Form covered | Notable features |
|---|---|---|
| _build_leave | Leave Application | 29-row table mirroring the Word doc; leave type checkboxes; HR-only section w |
| _build_commencement | Commencement | Employee joining info, bank details, reporting manager, dual signatures. |
| _build_duty_resumption | Duty Resumption | Leave dates, resumption date, manager remarks, three signatures (employee/H |
| _build_passport_release | Passport Release | Release/deposit toggle, purpose, date, three signatures. |
| _build_visa_renewal | Visa Renewal | Employee details, expiry, renewal date, three signatures. |
| _build_station_clearance | Station Clearance | Departure checklist with 14 checkbox items (equipment return, system access, |

| Function | Form covered | Notable features |
|---|---|---|
| _build_grievance | Grievance | Complainant details, grievance description, parties involved, HR + GM signatur |
| _build_interview_assessment | Interview Assessment | Rating matrix across competencies, recommendation, interviewer signature. |
| _build_staff_appraisal | Staff Appraisal | Detailed rating categories, strengths, improvement areas, overall score. |
| _build_performance_evaluation | Performance Evaluation | 10 scored criteria, evaluator + HR + GM signatures. |
| _build_contract_renewal | Contract Renewal | Four category rating blocks (5 criteria each), recommendation, three signatures |

## Entry points

```
# Called by module_hr/pdf_service.py
build_hr_pdf(form_type, form_data, output_stream, submission_id=None) → bool
# Looks up form_type in _BUILDERS dict, calls the _build_* function,
# wraps in SimpleDocTemplate(A4), doc.build(story) to output_stream.

supports_pdf(form_type) → bool
# Returns True if form_type is a key in _BUILDERS.
```

11

# HR DOCX Service — module_hr/docx_service.py

Produces **Word documents** by merging submission data into professionally designed **.docx template files** using **docxtpl** (Jinja2-style placeholders). Signature images are inserted as inline images at exact cell positions.

| Function | Description |
|---|---|
| _get_hr_documents_path() | Returns the path to the 'HR Documents - Main' template folder. |
| _get_template_path(form_type) | Locate the .docx template for a given form type; auto-detects whether it contains docxtpl placehold |
| _fmt_date(v) | Format date as DD/MM/YYYY; handles string, date, and datetime inputs. |
| _normalize_form_data_for_docx(form_data, form_type) | Map UI field names to template placeholder keys for each form type. Ensures every placeholder in |
| _build_generic_context(fd, form_type) | Build the full docxtpl context dict from form_data, applying date formatting and field normalization. |
| _add_signatures_to_context(doc, context, fd, form_type) | Decode each base64 signature in form_data into a docxtpl InlineImage and insert into the context |
| _signature_to_transparent_png(data_url) | Convert a base64 PNG/JPEG signature to a white-background PNG suitable for DOCX embedding |
| _signature_to_inline_image(doc, data_url, width_cm) | Return a docxtpl InlineImage from a base64 data URL. |
| _post_render_leave_checkboxes(doc, fd) | After docxtpl render, scan the document for leave-type checkbox placeholders and tick the correct |
| _post_render_leave_options_integrity(doc, fd) | Ensure leave checkboxes are consistent (exactly one ticked). |
| _post_render_leave_salary_advance(doc, fd) | Tick YES or NO for salary advance on the rendered document. |
| _post_render_interview_ratings(doc, fd) | Fill interview rating cells after template render. |
| _apply_post_render(doc, fd, form_type) | Dispatch to the appropriate post-render helpers for the given form type. |
| _generate_filled_docx_generic(submission, output_stream) | Main pipeline: get template → build context → render via docxtpl → post-render → write to stream |
| generate_commencement_docx(submission, output_stream) | Custom, form-specific generator with custom context and signature placement. |
| generate_performance_evaluation_docx(submission, output_stream) | Performance-evaluation-specific generator. |
| generate_hr_docx(submission, output_stream) | Entry point called by the download route. Selects the correct generator for the form type. |
| get_supported_docx_forms() | Returns list of form type strings that have a DOCX template. |
| fit_docx_to_one_page(doc) | Reduce margins and font sizes on the rendered document to keep it on a single page if possible. |

12

# HR Print Utilities — module_hr/print_utils.py

Generates **HTML fragments** suitable for browser printing or embedding in the **hr_print.html** template. The print view is available at **/hr/print/<submission_id>** and renders all fields plus embedded signature images.

| Function | Description |
|---|---|
| _esc(v) | HTML-escape a value for safe insertion into markup. |
| _fmt_date(v) | Format a date value as DD/MM/YYYY. |
| _row(label, value) | Return an HTML table row with a bold label cell and a value cell. |
| _sig_row(label, data_url) | Return an HTML row with an embedded base64 signature image (or blank if absent). |
| render_leave_print(fd, sid) | Build the complete Leave Application HTML table including all leave details and all four signature rows |
| render_generic_print(module_type, fd, sid) | Render all form_data key/value pairs (skipping hr_/gm_ internal keys) and render as a simple HTML table |
| render_form_for_print(module_type, fd, sid) | Dispatch: leaves → render_leave_print; grievance/passport/duty → specialised renderers; all others – |

13

# HVAC & MEP Module — module_hvac_mep/

Blueprint prefix: **/hvac-mep**. Handles HVAC and MEP site visit forms. Report generation (Excel + PDF) runs asynchronously via a background thread.

| Function / Route | Auth | Description |
|---|---|---|
| index  GET /hvac-mep/form | JWT | Render the HVAC form page. Checks access_hvac flag. |
| dropdowns  GET /hvac-mep/dropdowns | None | Return dropdown JSON (equipment types, observations). Cached if Redis avail |
| save_draft  POST /hvac-mep/save-draft | None | Save draft JSON to a local file keyed by a draft ID. |
| upload_photo  POST /hvac-mep/upload-photo | Optional JWT | Validate file (≤10MB, image type) → upload to Cloudinary → return URL. |
| submit  POST /hvac-mep/submit | Optional JWT | Validate payload → create_submission_db → create_job_db → executor.subm |
| status  GET /hvac-mep/status/<job_id> | None | Poll get_job_status_db(job_id) → return {status, progress, result_data}. |
| download_file  GET /hvac-mep/download/<job_id>/<type> | None | Stream Excel or PDF from cloud URL stored in job.result_data. |
| process_job(sub_id, job_id, …) | Background | Full pipeline via common.module_base.process_report_job: Excel → Cloudinar |

## Background report pipeline

```
# common/module_base.py — process_report_job()
mark_job_started(job_id) # progress = 10
submission = get_submission_db(sub_id)
excel_bytes = create_excel_report(submission) # progress = 30
excel_url = upload_file_to_cloud(excel_bytes) # progress = 45
pdf_bytes = create_pdf_report(submission) # progress = 60
pdf_url = upload_file_to_cloud(pdf_bytes) # progress = 75
complete_job_db(job_id, {excel: excel_url, pdf: pdf_url}) # 100
```

14

## Civil Works Module — module_civil/

Blueprint prefix: **/civil**. Identical structure to HVAC/MEP. Key routes: **GET /civil/form** (JWT), **POST /civil/submit** (optional JWT), **GET /civil/status/<job_id>**, **POST /civil/upload-photo**. Background pipeline uses the same **process_report_job** from common.

The Civil module's **civil_generators.py** implements **create_excel_report** and **create_pdf_report** using openpyxl/XlsxWriter and ReportLab respectively, with Civil-specific field layout and photo embedding.

15

## Cleaning Module — module_cleaning/

Blueprint prefix: **/cleaning**. Same pattern as HVAC and Civil. Notable difference: **submit_with_urls** requires JWT while plain **submit** accepts unauthenticated requests (for mobile offline scenarios). Background report pipeline is identical.

16

## Inspection Module — module_inspection/

Blueprint prefix: **/inspection**. A cross-trade wrapper. **_has_inspection_access(user)** returns True if the user has any of access_hvac, access_civil, or access_cleaning. The dashboard route redirects to the appropriate trade module or shows a combined view.

17

# Procurement Module — module_procurement/

Blueprint prefix: **/procurement**. All routes require JWT + access_procurement_module. Manages a materials catalogue, properties, and per-property material assignments.

| Function group | Routes | Description |
|---|---|---|
| Dashboard & pages | /procurement/, /materials, /add-material | HTML dashboard pages. |
| Materials CRUD | /api/materials (GET, POST, DELETE) | List, add, delete materials. Includes recent activity feed. |
| Excel import/export | /api/import-excel, /export-excel, /sample-excel | Bulk import from Excel; export current catalogue; download a sa |
| Properties | /api/properties (GET, POST) | List and create property records. |
| Property materials | /api/properties/<id>/materials (GET, POST) | List or assign materials to a specific property. |
| Catalog by department | /api/catalog/<dept> (GET, POST, PUT, DELETE) | Full CRUD on the materials catalog filtered by department. |
| Registered properties | /api/registered-properties (GET) | List all registered property records. |

18

# MMR Reporting — module_mmr/

Blueprint prefix: **/admin/mmr**. All routes require JWT. MMR handles CAFM data ingestion, analytics, report generation, scheduled email automation, and cycle lifecycle management.

## Internal helper functions

| Helper | Description |
|---|---|
| _load_config() / _save_config(cfg) | Read/write the MMR JSON config file (schedule, recipients, paths). |
| _env_schedule_override() | Read MMR schedule from environment variables — survives redeploys if the config file is ephe |
| _dashboard_payload_from_path(path) | Parse an uploaded CAFM file and return structured analytics payload for the dashboard. |
| _save_report_to_folder(bytes, name) | Write a generated report to the configured local reports folder. |
| _save_email_report_to_network(bytes, name) | Write report to a configured network/UNC path (optional Windows feature). |
| _save_report_to_drive(bytes, filename, path) | Save to a configurable drive/directory path. |
| _start_new_cycle / _approve_current_cycle / _complete_current_cycle | Lifecycle functions for the MMR cycle (upload → approve → email sent). |
| _load_cycle_log / _save_cycle_log | Read/write the JSON cycle log file. |
| append_automation_activity(action, status, detail) | Append a timestamped entry to the automation activity log. |
| _activities_for_cycle_id(cycle_id) | Return activity log entries matching a cycle. |
| _report_filename(date, range_str, format) | Build a standardised report filename including date and format suffix. |
| _resolve_report_format(explicit) | Determine output format (xlsx/pdf/csv) from request or config. |
| _validate_injaaz_emails(raw) | Parse a comma-separated email string; validate each address; return (valid_list, error_msg). |

## API route handlers

| Handler | Method | Description |
|---|---|---|
| dashboard | GET | Render MMR dashboard HTML page. |
| upload | POST | Accept CAFM file upload → parse → start new cycle → return dashboard payload. |
| current_upload | GET | Return metadata of the currently staged upload. |
| clear_upload | POST | Clear the staged upload file and reset cycle. |
| download_report | GET | Generate and stream the daily/custom date-range report. |
| download_report_monthly | GET | Generate and stream a monthly summary report. |
| save_to_folder | POST | Generate report and save to the local reports folder. |
| save_to_drive | POST | Generate and save to a configured network/drive path. |

| Handler | Method | Description |
|---|---|---|
| list_report_folder | GET | Return list of saved reports in the reports folder. |
| download_report_folder_file | GET | Stream a specific saved report file. |
| get_email_config | GET | Return current email schedule and recipients config. |
| save_email_config | POST | Update email config (schedule time, timezone, recipients). |
| send_email_now | POST | Immediately generate report and send to configured recipients. |
| get_automation_status | GET | Return scheduler running/paused state, last run, next run, last result. |
| pause_automation | POST | Pause the APScheduler job. |
| resume_automation | POST | Resume the APScheduler job. |
| get_cycles | GET | Return list of all reporting cycles. |
| approve_cycle | POST | Mark the current cycle as approved. |
| get_automation_activities | GET | Return automation activity log (last N entries). |

## Scheduler integration

```
# APScheduler is initialised in Injaaz.py create_app()
# MMR module registers a daily job when the config enables automation.
# Key config keys (from _load_config()):
# schedule_enabled: true/false
# schedule_hour: 10 # local time in schedule_timezone
# schedule_minute: 0
# schedule_timezone: 'Asia/Dubai' # default
# recipients: ['ops@company.com', ...]
# Environment overrides (survive redeploys when config file is ephemeral):
# MMR_SCHEDULE_ENABLED, MMR_SCHEDULE_HOUR, MMR_SCHEDULE_MINUTE, MMR_SCHEDULE_TIMEZONE
```

19

# Business Development — app/bd/routes.py

Blueprint prefix: **/bd**. All routes require JWT. Provides an email composition module that allows BD users to select completed submissions and email them with their generated report attachments to the GM or other recipients.

| Helper | Description |
|---|---|
| _is_bd_user(user) | True if user has BD designation or is admin. |
| _parse_emails(raw) | Split comma-separated email string into list. |
| _collect_submission_attachments(submission_ids) | Gather Excel/PDF URLs from Job.result_data for a list of submission IDs. |
| _get_report_urls(submission) | Extract excel_url and pdf_url from the submission's most recent completed Job. |
| _get_gm_emails() | Query all users with designation=general_manager and return their email addresses. |
| _get_role_emails(role) | Query users by role and return emails. |

| Route | Method | Description |
|---|---|---|
| email_module  /bd/email-module | GET | Render the BD email composition page. |
| list_email_attachments  /bd/email-module/attachments | GET | Return all completed submissions with their Excel/PDF attachment URL |
| download_attachment  /bd/email-module/attachment/<id>/<type> | GET | Stream one attachment file (Excel or PDF) from Cloudinary. |
| send_email_to_gm  /bd/email-module/send | POST | Compose and send email with selected submission attachments to GM |

20

# DocHub — app/docs/routes.py

Blueprint prefix: **/api/docs**. Most routes use **token_required** (JWT + valid non-revoked Session + is_active user). The inline image serve endpoint is intentionally public but uses unguessable UUID filenames.

| Function | Method | Path | Description |
|---|---|---|---|
| access_check | GET | /api/docs/access-check | Verify the caller has DocHub access (DocHubAccess flag s |
| list_documents | GET | /api/docs | Return all documents the caller is allowed to see. |
| create_document | POST | /api/docs | Create a new content-type document with title and body. |
| upload_documents | POST | /api/docs/upload | Upload one or more file-type documents (PDF, DOCX, etc. |
| get_document | GET | /api/docs/<id> | Return full document including body/refs. |
| download_document | GET | /api/docs/<id>/download | Stream document file; DOCX converted to PDF via LibreOf |
| preview_upload_as_pdf | GET | /api/docs/<id>/preview | Return a PDF preview of the document (conversion via Lib |
| update_document | PATCH | /api/docs/<id> | Update title, body, or file attachment. |
| delete_document | DELETE | /api/docs/<id> | Delete document record and cloud file. |
| upload_inline_editor_image | POST | /api/docs/inline-image | Upload image pasted into the rich-text editor; return local s |
| serve_inline_editor_image | GET | /api/docs/inline/<filename> | No-auth serve of inline editor images (UUID filenames prev |
| upload_inline_reference | POST | /api/docs/inline-reference | Attach a reference file to a document. |
| _has_dochub_access(user) | — | — | Returns True if DocHubAccess row exists for user, or user |
| _docx_to_pdf_cached(path) | — | — | Convert a .docx to PDF using LibreOffice if available; cach |

21

# Reports API — app/reports_api.py

Blueprint prefix: **/api/reports**. JWT-protected. Allows on-demand regeneration of Excel and PDF reports for Civil, HVAC, and Cleaning submissions without re-submitting the form.

| Function | Method | Path | Description |
|---|---|---|---|
| _user_can_access_submission(user, sub) | — | | True if admin, submission owner, or workflow par |
| regenerate_excel | GET | /api/reports/regenerate/<id>/excel | Re-run create_excel_report for the module; strea |
| regenerate_pdf | GET | /api/reports/regenerate/<id>/pdf | Re-run create_pdf_report; stream result. |
| list_reports | GET | /api/reports/list/<module_type> | Return submissions for a module with their downl |
| get_submission_details | GET | /api/reports/submission/<id> | Return submission detail including regenerate lin |

22

# Security Architecture

## Authentication flow (JWT + sessions)

```
Every protected route:
@jwt_required() # Flask-JWT-Extended decorator
↓
JWTManager checks token signature + expiry
↓
token_in_blocklist_loader(jti) called:
Session.query.filter_by(token_jti=jti).first()
→ None or is_revoked=True → reject (401)
→ found and not revoked → allow
↓
get_jwt_identity() → user_id string
User.query.get(user_id) → user object
```

## Password security

```
# On register / password change:
user.set_password(plain_text)
→ bcrypt.generate_password_hash(plain_text).decode('utf-8')
→ stored in user.password_hash

# On login:
user.check_password(plain_text)
→ bcrypt.check_password_hash(user.password_hash, plain_text)
→ True/False (original plain text never stored)
```

## Rate limiting

**Flask-Limiter** is applied to the login and registration endpoints. Default limit: **5 requests per minute**. If Redis is configured, limits are tracked across workers; otherwise in-memory (single worker only).

## Path traversal prevention

```
# common/security.py
def safe_path_join(base_dir, *parts) -> str | None:
joined = os.path.realpath(os.path.join(base_dir, *parts))
if not joined.startswith(os.path.realpath(base_dir)):
log_security_event('path_traversal_attempt', ...)
return None
return joined
```

23

# End-to-End Request Flows

## Flow A — Employee submits a Leave Application

```
1. Employee logs in → POST /api/auth/login → JWT access token stored in cookie
2. Employee opens /hr/leave-application-form → hr_leave_application_form() renders HTML
3. Employee fills form, draws signature in canvas (stored as base64 data URL)
4. POST /hr/api/submit {form_type:'leave_application', employee_name:…,
employee_signature:'data:…', …}
→ submit_hr_form():
submission_id = 'HR-LEAVE_APPLICATION-' + uuid4().hex[:8].upper()
Submission(module_type='hr_leave_application', workflow_status='hr_review', form_data={…})
db.session.commit()
create_notification(hr_user_id, 'New HR Request', …) for each HR user
← {success:true, submission_id:'HR-LEAVE_APPLICATION-XXXX'}
5. HR Officer opens /hr/pending-review → get_pending_hr_review() → list of hr_review
submissions
6. POST /hr/api/hr-approve/HR-LEAVE_APPLICATION-XXXX
{signature:'data:…', comments:'Approved', form_data_hr:{hr_checked:'yes', hr_balance_cf:'10',
…}}
→ hr_approve():
form_data = dict(submission.form_data) # copy
form_data['hr_signature'] = 'data:…'
submission.workflow_status = 'gm_review'
db.session.commit()
create_notification(gm_user_id, 'Pending Your Approval', …)
← {success:true, message:'Forwarded to GM'}
7. GM opens /hr/gm-approval → get_pending_gm_approval()
8. POST /hr/api/gm-approve/HR-LEAVE_APPLICATION-XXXX {signature:'data:…', comments:'Approved'}
→ gm_approve():
form_data['gm_signature'] = 'data:…'
submission.workflow_status = 'approved'; submission.status = 'completed'
create_notification(employee_id, 'Request Approved', …)
create_notification(hr_reviewer_id, 'Approved by GM', …)
← {success:true}
9. Employee downloads document:
GET /hr/download-pdf/HR-LEAVE_APPLICATION-XXXX
→ hr_download_pdf() → generate_hr_pdf(submission, buf) → build_hr_pdf('leave_application',
…)
← stream PDF with all three signatures embedded
```

## Flow B — Field inspector submits HVAC/MEP report

```
1. Inspector opens /hvac-mep/form (JWT checked, access_hvac verified)
2. GET /hvac-mep/dropdowns ← cached equipment/observation options
3. Inspector uploads photos: POST /hvac-mep/upload-photo (each photo)
```

```
→ validate_file_upload() → upload_to_cloudinary_with_retry() → return {url}
4. Inspector submits: POST /hvac-mep/submit {site_name, visit_date, form_data:{items:[…]}}
→ create_submission_db() → Submission(status='submitted')
→ create_job_db(sub_id) → Job(status='pending')
→ executor.submit(process_job, sub_id, job_id, config, app)
← {job_id: 'job_abc123'}
5. Frontend polls: GET /hvac-mep/status/job_abc123 every 2 seconds
← {status:'processing', progress:45}
6. Background thread (process_report_job):
create_excel_report(submission) → .xlsx bytes
upload_file_to_cloud(.xlsx) → excel_url
create_pdf_report(submission) → .pdf bytes (photos fetched from Cloudinary)
upload_file_to_cloud(.pdf) → pdf_url
complete_job_db(job_id, {excel:excel_url, pdf:pdf_url}) # progress=100
7. Frontend receives: {status:'completed', result_data:{excel:…, pdf:…}}
→ show download links
8. GET /hvac-mep/download/job_abc123/pdf ← stream PDF
```

24

# Function & Method Quick Reference

A consolidated index of key functions across the codebase for quick lookup.

| Function | File | Purpose |
|---|---|---|
| create_app() | Injaaz.py | Flask application factory; registers all blueprints and extens |
| validate_config(app) | common/config_validator.py | Startup config check; raises on critical failures. |
| sync_access_session_row(jti, payload) | common/jwt_session.py | Ensure a JWT session row exists (blocklist safety). |
| error_response(msg, code, …) | common/error_responses.py | Standardised JSON error body. |
| success_response(data, msg) | common/error_responses.py | Standardised JSON success body. |
| random_id(prefix) | common/utils.py | Generate unique submission/job IDs. |
| upload_file_to_cloud(path) | common/utils.py | Cloudinary upload returning secure URL. |
| upload_base64_to_cloud(data_url) | common/utils.py | Base64 data URL → Cloudinary. |
| get_image_for_pdf(url_or_b64) | common/utils.py | Fetch/decode image for ReportLab embedding. |
| safe_path_join(base, *parts) | common/security.py | Path traversal guard. |
| send_email(to, subject, html, …) | common/email_service.py | Multi-provider email send (Brevo/Mailjet/SMTP). |
| process_report_job(…) | common/module_base.py | Standard HVAC/Civil/Cleaning background job pipeline. |
| validate_request(schema, data) | common/validation.py | Marshmallow schema validation helper. |
| cached(key, ttl) decorator | common/cache.py | Redis-backed function result caching. |
| retry_on_failure(fn, n) | common/retry_utils.py | Re-call fn up to n times on exception. |
| create_submission_db(…) | common/db_utils.py | Insert Submission ORM row. |
| complete_job_db(job_id, result) | common/db_utils.py | Mark Job completed with result URLs. |
| User.set_password(pw) | app/models.py | Bcrypt hash and store password. |
| User.has_module_access(module) | app/models.py | Permission check respecting admin bypass. |
| Submission.to_dict(…) | app/models.py | API serialisation of a submission. |
| login() | app/auth/routes.py | Credential verification + JWT issuance + session row. |
| logout() | app/auth/routes.py | Session revocation + cookie clear. |
| hr_approve(submission_id) | module_hr/routes.py | HR sign-off; advance to gm_review stage. |
| gm_approve(submission_id) | module_hr/routes.py | GM final approval; mark approved/completed. |
| create_notification(user_id, …) | module_hr/routes.py | Insert Notification row for one user. |

| Function | File | Purpose |
|---|---|---|
| generate_hr_docx(submission, stream) | module_hr/docx_service.py | Entry point: pick template + merge + signatures. |
| generate_hr_pdf(submission, stream) | module_hr/pdf_service.py | Entry point: normalise data + build ReportLab PDF. |
| build_hr_pdf(form_type, fd, stream) | module_hr/hr_pdf_builder.py | Dispatch to _build_* function + doc.build(). |
| _sig_to_image(data_url, w, h) | module_hr/hr_pdf_builder.py | Base64 signature → ReportLab Image. |
| render_form_for_print(module, fd, id) | module_hr/print_utils.py | Build print-ready HTML for any HR form. |
| _normalize_form_data_for_docx(fd, ft) | module_hr/docx_service.py | Map UI keys to DOCX template placeholder keys. |
| upload() | module_mmr/routes.py | Accept CAFM file, parse, start new reporting cycle. |
| send_email_now() | module_mmr/routes.py | Immediately generate and email the MMR report. |
| _load_config() | module_mmr/routes.py | Read MMR schedule + email config JSON file. |
| _start_new_cycle(uploaded_by) | module_mmr/routes.py | Open a new reporting cycle in the cycle log. |
| append_automation_activity(…) | module_mmr/routes.py | Record automation event in activity log. |
| send_email_to_gm() | app/bd/routes.py | Compose and send email with report attachments. |
| download_document() | app/docs/routes.py | Stream DocHub document; DOCX→PDF if needed. |
| regenerate_excel/pdf() | app/reports_api.py | On-demand report regeneration without re-submit. |

*This document was generated on 09 April 2026. It reflects the full function and method surface of the Injaaz platform as implemented. Re-run 'python scripts/generate_full_technical_doc.py' to refresh after code changes.*